

Practica 2 Programación Orientada a Aspectos:

Guía Práctica: Ciberseguridad con Programación Orientada a Aspectos (POA)

Materia: Programación Orientada a Aspectos

Proyecto: SentinelAuth - Detección de Intrusos mediante Aspectos en Spring Boot.

1. Configuración del Proyecto (Spring Initializr)

Acceder a start.spring.io y configurar:

- **Project:** Maven
 - **Language:** Java 17+
 - **Spring Boot:** 3.2.x
 - **Artifact:** sentinel-auth
 - **Dependencies:**
 - Spring Web
 - Spring Boot Starter AOP
 - Lombok
 - Spring Security
-

2. Preparación del "Terreno" (Lógica de Negocio)

Primero, definimos nuestra clase de datos (DTO) y el controlador. Este código es "limpio", no sabe que lo están vigilando.

DTO: `LoginRequest.java`

Java

```
public record LoginRequest(String username, String password) {}
```

Controlador: `AuthController.java`

Java

```
@RestController
@RequestMapping("/auth")
public class AuthController {

    @PostMapping("/login")
    public String login(@RequestBody LoginRequest request) {
        // Lógica de negocio pura
    }
}
```

```

        if ("admin".equals(request.username()) &&
"root123".equals(request.password())) {
            return "Token-JWT-Exitoso";
        }
        throw new RuntimeException("Credenciales Inválidas");
    }
}

```

3. Implementación del Aspecto (El Centinela)

Aquí aplicamos los conceptos de tu temario: **Aspecto, Pointcut y Advice**.

Clase: SecuritySentinelAspect.java

Java

```

@Aspect
@Component
public class SecuritySentinelAspect {

    private static final Logger auditLog =
LoggerFactory.getLogger("SecuritySentinel");
    private final Map<String, Integer> failedAttempts = new
ConcurrentHashMap<>();
    private final Map<String, Long> lastRequestTime = new
ConcurrentHashMap<>();

    // PUNTO DE CORTE (Pointcut): Vigilamos el método login
    @Pointcut("execution(*
com.example.demo.AuthController.login(..))")
    public void loginPointcut() {}

    // CONSEJO BEFORE (Advice): Detección de DoS
    @Before("loginPointcut()")
    public void detectDoS() {
        ServletRequestAttributes attr = (ServletRequestAttributes)
RequestContextHolder.currentRequestAttributes();
        String ip = attr.getRequest().getRemoteAddr();
        long now = System.currentTimeMillis();

        if (lastRequestTime.containsKey(ip) && (now -
lastRequestTime.get(ip) < 200)) { // 200ms entre peticiones
            logAttack("DENIAL_OF_SERVICE", ip, "Frecuencia excesiva",
attr.getRequest());
            throw new
ResponseStatusException(HttpStatus.TOO_MANY_REQUESTS, "Ataque
detectado");
        }
        lastRequestTime.put(ip, now);
    }

    // CONSEJO AFTER THROWING (Advice): Fuerza Bruta
    @AfterThrowing(pointcut = "loginPointcut()", throwing = "ex")
    public void detectBruteForce(JoinPoint jp) {
        ServletRequestAttributes attr = (ServletRequestAttributes)
RequestContextHolder.currentRequestAttributes();
        String ip = attr.getRequest().getRemoteAddr();
        LoginRequest dto = (LoginRequest) jp.getArgs()[0];
    }
}

```

```

        int attempts = failedAttempts.getOrDefault(ip, 0) + 1;
        failedAttempts.put(ip, attempts);

        if (attempts >= 3) {
            logAttack("BRUTE_FORCE", ip, "Usuario intentado: " +
                dto.username(), attr.getRequest());
        }
    }

    private void logAttack(String type, String ip, String detail,
        HttpServletRequest req) {
        String logEntry = String.format(
            "| TIPO: %s | IP: %s | RECURSO: %s | DETALLE: %s | AGENTE: %s |",
            type, ip, req.getRequestURI(), detail,
            req.getHeader("User-Agent")
        );
        auditLog.warn(logEntry);
    }
}

```

4. El Tejedor y el Registro (Logging)

Para que el **Tejedor** envíe la información al archivo físico, configuramos `src/main/resources/logback-spring.xml`:

XML

```

<configuration>
    <appender name="FILE" class="ch.qos.logback.core.FileAppender">
        <file>logs/seguridad_audit.log</file>
        <encoder>
            <pattern>%d{yyyy-MM-dd HH:mm:ss} %msg%n</pattern>
        </encoder>
    </appender>

    <logger name="SecuritySentinel" level="WARN">
        <appender-ref ref="FILE" />
    </logger>
</configuration>

```

5. Dinámica de Prueba: "El Hacker en la Terminal"

Una vez que el proyecto esté corriendo (`mvn spring-boot:run`), los alumnos deben simular los ataques.

Ataque 1: Fuerza Bruta

Ejecutar esto 3 veces con datos erróneos:

Bash

```

curl -X POST http://localhost:8080/auth/login \
-H "Content-Type: application/json" \
-d '{"username":"victima", "password":"wrong_password"}'

```

Ataque 2: DoS (Ráfaga rápida)

Bash

```
for i in {1..10}; do curl -X POST http://localhost:8080/auth/login -H  
"Content-Type: application/json" -d '{"username":"admin",  
"password":"123"}'; done
```

6. Resultados Esperados

Al terminar, debe existir un archivo en la raíz llamado `logs/seguridad_audit.log` con este formato:

```
2026-03-09 10:15:30 | TIPO: BRUTE_FORCE | IP: 127.0.0.1 | RECURSO:  
/auth/login | DETALLE: Usuario intentado: victima | AGENTE:  
curl/7.68.0 |
```
